



Facultad de Ingeniería
Departamento de Ingeniería en Computación e
Informática

Análisis e interpretación: Algoritmos recursivos de búsqueda y ordenamiento

Ayrton Morrison

Iván Gallardo

Pablo Gómez

Ingeniería Civil en Computación e Informática

Docente: MSc. Jackeline Aldridge

Ramo: Diseño de Algoritmos

RESUMEN

Este informe presenta el desarrollo de un sistema en lenguaje C para la gestión de información de deportistas, capaz de generar, almacenar, ordenar y buscar registros en archivos CSV. Para ello, se implementaron los algoritmos de ordenamiento Bubble Sort optimizado, Insertion Sort, Selection Sort optimizado y Cocktail Shaker Sort, junto con los algoritmos de búsqueda secuencial y búsqueda binaria iterativa. Además, se realizaron experimentos para analizar el comportamiento teórico y empírico de estos algoritmos en distintos tamaños de entrada y en diferentes casos de ejecución. Los resultados obtenidos permitieron comparar su rendimiento mediante benchmarks y gráficos, facilitando la comprensión de sus diferencias en eficiencia y complejidad.

ÍNDICE GENERAL

ÍNDICE DE FIGURAS

INTRODUCCIÓN

"**Divide y Vencerás**" es un paradigma de programación, que consiste en un conjunto de técnicas algorítmicas en las que primero se divide el problema a resolver en subproblemas de menor tamaño y de la misma naturaleza, luego se resuelven los subproblemas de manera independiente y recursiva, hasta llegar a un caso base (que es un caso tan trivial que detiene la recursión), y finalmente se combinan los resultados para llegar a la solución final.

En el presente informe, se analizará empíricamente la complejidad algorítmica de varios algoritmos que utilizan este paradigma de programación, y además, se evaluará el impacto que tiene cada variante de cada algoritmo.

Los algoritmos que serán objeto de este análisis involucrarán tanto algoritmos de ordenamiento, como algoritmos de búsqueda, como también un algoritmo de selección.

Estos algoritmos son:

Ordenamiento

- *Merge Sort*.
- *Merge-Insertion Sort* (versión de *Merge Sort* que incluye *Insertion Sort* cuando se llegan a tener subarreglos de tamaño pequeño).
- *Quick Sort* con pivote en la primera posición.
- *Quick Sort* con pivote en la última posición.
- *Quick Sort* con pivote en una posición aleatoria.
- *Quick Sort* escogiendo al pivote como resultado de una mediana entre 3 elementos.

Búsqueda

- Búsqueda binaria clásica.
- Búsqueda binaria para rangos.
- Búsqueda exponencial.

- Búsqueda por interpolación.

Selección

- *Quick Select*.

Todo lo anterior se encuentra enmarcado en un sistema de gestión de deportistas desarrollado previamente, el cual trabaja con registros que contienen información como ID, nombre, equipo, puntaje y cantidad de competencias disputadas. Sobre este conjunto de datos, se implementaron distintas funcionalidades prácticas, tales como el ordenamiento de deportistas según distintos criterios, la búsqueda eficiente de registros, la obtención del k-ésimo mejor deportista mediante algoritmos de selección, y la generación de rankings basados en puntaje. Además, se realizaron *benchmarks* experimentales utilizando distintos tamaños de entrada, con el objetivo de comparar el comportamiento y el rendimiento de los algoritmos implementados en escenarios de mejor caso, peor caso y caso promedio.

OBJETIVOS

2.1. Objetivo Principal

2.2. Objetivos Secundarios

MARCO TEÓRICO

3.1. Algoritmos de ordenamiento

Existen muchísimos algoritmos de ordenamiento, desde algunos muy eficientes, que pueden llegar a ser de una complejidad temporal de apenas $O(n)$, como *Radix Sort*, hasta algunos extremadamente ineficientes, que pueden llegar incluso a ser de $O(n \times n!)$, como lo es el caso de *Bogo Sort*, pero en este trabajo nos centraremos principalmente en *Merge Sort* y *Quick Sort*, que son algoritmos relativamente eficientes y que emplean el principio de **"Divide y Vencerás"**.

3.1.1. Merge Sort

Merge Sort, también conocido como "ordenamiento por Mezcla", es un algoritmo de ordenamiento recursivo, que en cada paso va dividiendo el arreglo en 2 subarreglos de aproximadamente la mitad del tamaño anterior, y eso lo hace hasta que todos los subarreglos sean de tamaño 1. Cuando eso sucede, se comienzan a combinar pares de subarreglos adyacentes, recorriéndolos linealmente para fusionarlos en el orden correcto, hasta que finalmente todo el arreglo queda ordenado.

En base al principio en el que se fundamenta *Merge Sort*, es posible deducir su complejidad temporal mediante la siguiente ecuación de recurrencia:

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n)$$

Esto se debe a que el problema se divide recursivamente en 2 subproblemas de tamaño $n/2$, mientras que el costo adicional corresponde al proceso de fusión de los subarreglos, el cual requiere recorrer linealmente los elementos para ubicarlos en el orden correcto, por lo que dicho costo es de complejidad lineal.

Para resolver la ecuación de recurrencia, se puede emplear el método de iteración. Considerando una constante k para representar el costo lineal, se tiene:

$$T(n) = 2T\left(\frac{n}{2}\right) + kn$$

A partir de esto, la expresión equivalente para $T(\frac{n}{2})$ sería:

$$T(\frac{n}{2}) = 2T(\frac{n}{4}) + k(\frac{n}{2})$$

Entonces, reemplazando en la ecuación original, se obtiene:

$$T(n) = 2 \left(2T(\frac{n}{4}) + k(\frac{n}{2}) \right) + kn$$

Simplificando:

$$T(n) = 4T(\frac{n}{4}) + 2kn$$

Ahora, realizando otra iteración, se tiene que:

$$T(\frac{n}{4}) = 2T(\frac{n}{8}) + k(\frac{n}{4})$$

Reemplazando nuevamente:

$$T(n) = 4 \left(2T(\frac{n}{8}) + k(\frac{n}{4}) \right) + 2kn$$

$$T(n) = 8T(\frac{n}{8}) + 3kn$$

Continuando de esta forma, se puede generalizar que:

$$T(n) = 2T(\frac{n}{2}) + kn = 4T(\frac{n}{4}) + 2kn = 8T(\frac{n}{8}) + 3kn = 2^i T(\frac{n}{2^i}) + ikn$$

La recursión finaliza cuando se alcanza el caso base, es decir, cuando los subarreglos tienen tamaño 1. Por lo tanto, debe cumplirse que:

$$\frac{n}{2^i} = 1$$

Despejando:

$$i = \log_2(n)$$

Reemplazando esto en la expresión general obtenida anteriormente:

$$T(n) = 2^{\log_2(n)}T(1) + \log_2(n)kn$$

Como $2^{\log_2(n)} = n$, entonces:

$$T(n) = nT(1) + kn \log_2(n)$$

Finalmente, considerando que $T(1)$ y k son constantes, se concluye que:

$$T(n) \in O(n \log n)$$

Por lo tanto, *Merge Sort* posee complejidad temporal $O(n \log n)$ tanto en el mejor caso, como en el caso promedio y peor caso, ya que el arreglo siempre se divide de manera uniforme, independientemente del orden inicial de los elementos.

3.1.2. Merge-Insertion Sort

Merge-Insertion Sort corresponde a una variante optimizada de *Merge Sort*, cuyo objetivo es reducir el overhead recursivo que se produce al trabajar con subarreglos muy pequeños.

La idea principal consiste en que, cuando los subarreglos alcanzan un tamaño suficientemente pequeño, en lugar de continuar dividiéndolos recursivamente, se utiliza *Insertion Sort* para ordenarlos directamente.

Esto se hace debido a que *Insertion Sort*, a pesar de poseer complejidad $O(n^2)$ en el peor caso, tiene un rendimiento muy eficiente para arreglos de tamaño reducido, ya que posee un overhead muy bajo y además aprovecha favorablemente arreglos parcialmente ordenados.

De esta manera, *Merge-Insertion Sort* busca combinar las ventajas de ambos algoritmos: la eficiencia asintótica de *Merge Sort* para arreglos grandes, junto con la simplicidad y rapidez de *Insertion Sort* en subarreglos pequeños.

En términos de complejidad asintótica, esta variante sigue perteneciendo a $O(n \log n)$, aunque en la práctica puede presentar mejoras de rendimiento respecto a la versión clásica

de *Merge Sort*, dependiendo del umbral utilizado para decidir cuándo aplicar *Insertion Sort*, que es uno de los aspectos que se analizará en el presente trabajo.

3.1.3. Quick Sort

DESARROLLO

4.1. Diseño general del sistema

El sistema fue diseñado para gestionar información de deportistas de forma simple y modular, permitiendo generar datos, cargarlos desde archivos CSV y aplicar sobre ellos distintas operaciones de ordenamiento, búsqueda y análisis experimental. Para facilitar su desarrollo y comprensión, la implementación se organizó en componentes que separan la representación de los datos, la generación y carga de registros, y las funcionalidades principales ofrecidas por el programa.

4.1.1. Estructura de datos de los deportistas

La estructura de datos utilizada para representar a cada deportista se definió como una estructura con campos específicos para almacenar distinta información; estos campos corresponden a:

- **ID:** Identificador único del deportista.
- **Nombre:** Nombre completo del deportista.
- **Equipo:** Equipo al que pertenece.
- **Puntaje:** Puntuación total acumulada.
- **Competencias:** Número de competencias en las que ha participado.

Cada parámetro permite almacenar información, la cual puede ser utilizada para ordenar o buscar a los deportistas según distintos criterios. Esta estructura se implementó utilizando un `struct` en C, lo que facilita su manejo y acceso a los campos correspondientes durante las operaciones de ordenamiento y búsqueda.

4.1.2. Generación y carga de datos

La generación de datos de los parámetros asociados a deportistas se realizó mediante distintas funciones, las cuales permiten crear u obtener estos registros de manera pseudo-aleatoria. La generación de un nombre se realizó permitiendo seleccionar aleatoriamente letras de un conjunto predefinido, escogiendo un tamaño de 3 a 8 caracteres. Para el equipo,

se seleccionó aleatoriamente entre un conjunto específico de equipos de Esports. El ID se generó en forma incremental, asegurando la unicidad de cada registro. Por otro lado, el puntaje y la cantidad de competencias se generaron utilizando funciones de generación de números aleatorios dentro de rangos predefinidos. Esta metodología permitió crear un conjunto de datos variado y representativo para probar las funcionalidades del sistema. Posterior a la creación de los registros, estos se desordenaron utilizando el algoritmo de Fisher-Yates para garantizar que los datos no se encuentren en un orden específico, lo que es fundamental para evaluar correctamente el desempeño de los algoritmos de ordenamiento y búsqueda implementados. Finalmente, los registros generados se almacenaron en un archivo CSV, lo que facilita su posterior carga y manipulación dentro del programa.

CONCLUSIÓN

ANEXOS

6.1. Código fuente

6.2. Contribución por integrante